



信息学奥林匹克竞赛

Olympiad in Informatics

动态规划—优化+进阶应用

陈 真

上海市实验学校

动态规划进阶应用

进阶应用1：动态规划+二分

【题目01】[POI 2013] LUK-Triumphal arch (洛谷P3554)

【题目描述】

给一颗 n ($n \leq 3 \times 10^5$) 个节点的树，初始时1号节点被染黑，其余是白的。两个人轮流操作，一开始 B 在 1号节点。每一轮，A 选择 k 个点染黑，然后 B 走到一个相邻节点，如果 B 当前处于白点则 B 胜，否则当 A 将所有点染为黑点时 A 胜。求能让 A 获胜的最小的 k 。

【输入格式】

第一行读入一个整数 n ，表示树的节点数

接下来 $n-1$ 行，每行读入两个用空格分隔的整数 u,v ，表示 u 、 v 之间有一条边。

【输出格式】

输出仅有一行，表示让 A 获胜的最小的 k 。

【输入样例1】 【输出】

```
7
3
1 2
1 3
2 5
2 6
7 2
4 1
```

动态规划进阶应用

进阶应用1：动态规划+二分

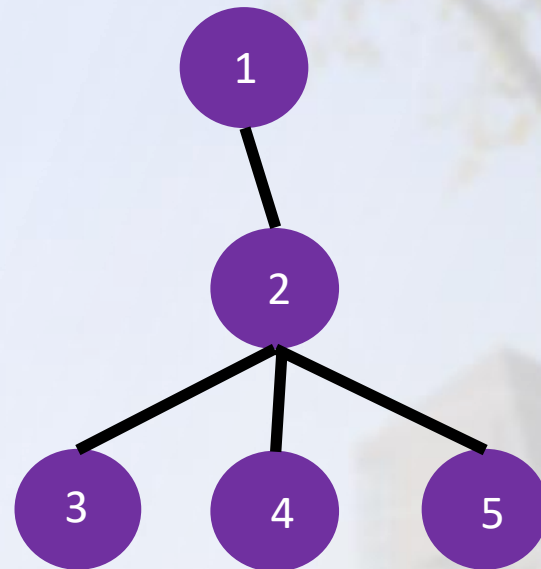
【题目01】 [POI 2013] LUK-Triumphal arch (洛谷P3554)

【分析】 k满足一定的单调性，而且有上下界。（二分答案？）

$dp[i]$ 为在i号节点的子树中还需要多染色的点。

给每一结点的儿子们染色时如果可染色数大于这一结点的儿子数
则要继续下去给儿子的儿子染色

$$dp[x] = \max(\sum dp[y] + \text{son}[x] - k, 0) \quad \text{若} dp[1]=0 \text{则答案} k \text{可行。}$$



二分区间收缩:

可行: 往更小找;

不可行: 往更大找;

```
34 while (l<r) {  
35     memset(dp,0,sizeof(dp));  
36     mid=(l+r)>>1;  
37     dfs(1,0);  
38     if (!dp[1])r=mid;  
39     else l=mid+1;  
40 }
```

```
10 void dfs(int u,int fa){  
11     int sum=0;  
12     for(int i=head[u];i;i=edge[i].nxt){  
13         int v=edge[i].v;  
14         if(v==fa)continue;  
15         dfs(v,u);  
16         sum+=dp[v]+1;  
17     }  
18     dp[u]=max(sum-mid,0);  
19 }
```

动态规划进阶应用

进阶应用1：动态规划+二分

【总结】常见应用总结限制是变量，无法直接 DP，只能枚举限制（二分），每次用 DP 校验可行性。

二分一个限制值 mid （最大值 / 最小值 / 代价 / 容量 / 距离）；判定：是否存在合法方案满足限制 $\leq mid / \geq mid$ ；动态规划通常相当于充当二分里的 $check(mid)$ 校验函数，快速求出限制为 mid 时的最优 / 可行状态，判断是否达标。

推荐题目

序号	题目	备注
1	[NWRRC 2015] Journey to the “The World’s Start”	P7057
2	software	P1800
3	[CEOI 2024] 洒水器	P10806

动态规划进阶应用

进阶应用2：动态规划+倍增

【题目02】跑路 (P1613)

【题目描述】

小 A 的工作不仅繁琐，更有苛刻的规定，要求小 A 每天早上在 6:00 之前到达公司，否则这个月工资清零。可是小 A 偏偏又有赖床的坏毛病。于是为了保住自己的工资，小 A 买了一个空间跑路器，每秒钟可以跑 2^k 千米（ k 是任意自然数）。当然，这个机器是用 32 位整数存的，所以总跑路长度不能超过 $2^{31}-1$ 千米。小 A 的家到公司的路可以看做一个有向图，小 A 家为点 1，公司为点 n ，每条边长度均为一千米。小 A 想每天能醒地尽量晚，所以让你帮他算算，他最少需要几秒才能到公司。数据保证 1 到 n 至少有一条路径。

【输入格式】

第一行两个整数 n, m ，表示点的个数和边的个数。

接下来 m 行每行两个数字 u, v ，表示一条 u 到 v 的边。

【输出格式】

一行一个数字，表示到公司的最少秒数。

【样例解释】 $1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$ ，总路径长度为 4 千米，直接使用一次跑路器即可。

【数据范围】

50% 的数据满足最优解路径长度 ≤ 1000 ；

100% 的数据满足 $2 \leq n \leq 50$ ， $m \leq 10^4$ ，最优解路径长度 $\leq 2^{31}-1$ 。

【输入样例1】

```
4 4
1 1
1 2
2 3
3 4
```

【输出样例1】

```
1
```

动态规划进阶应用

进阶应用2：动态规划+倍增

【题目02】跑路 (P1613)

【分析】

- “空间跑路器，每秒钟可以跑 2^k 千米 (k 是任意自然数) ”，也就是任意 2^k 的路径长度都可以在1秒内走完。
- 如果有 $u \rightarrow v$ 两点间的最短距离为 2^k 千米，即可认为这两点之间有一条边，边权为1。

令 $dp_{i,j,k}$ 的意义：能否在 2^i 时间内从点 j 走到点 k ，可以为1，可在 j 到 k 之间建一条边，否则为0。

$$dp_{i,j,k} = \sum (dp_{i-1,j,t} \&\& dp_{i-1,t,k}) ,$$

预处理完成之后，在跑一个 Floyd 最短路 ($n < 50$)。

动态规划进阶应用

进阶应用2：动态规划+倍增

【总结】动态规划与倍增结合常见应用时，需要预处理倍增表，再拆分答案二进制求解，而且DP的转移要满足“**转移运算满足结合律**”。连续两次操作能否合并为等价一次操作，信息可叠加 / 拼接。

通常设DP状态 $dp[t][u]$ 从状态 u ，执行 2^t 次基础操作后得到的结果 / 终点 / 区间信息。

推荐题目

序号	题目	备注
1	[USACO16OPEN] 262144 P	P3147
2	[POI 2010] ZAB-Frog	P3509
3	[KOI 2025 #2] 机器人	P13522

动态规划进阶应用

进阶应用3：动态规划+基环树

【题目03】创世纪（P10933）

【题目描述】

上帝手中有 N 种世界元素，每种元素可以限制另外1种元素，把第 i 种世界元素能够限制的那种世界元素记为 $A[i]$ 。现在，上帝要把它们中的一部分投放到一个新的空间中去建造世界。为了世界的和平与安宁，上帝希望所有被投放的世界元素都至少有一个能够限制它的世界元素没有被投放。上帝希望知道，在此前提下，他最多可以投放多少种世界元素？

【输入格式】

第一行是一个整数 N ，表示世界元素的数目。

第二行有 N 个整数 $A[1], A[2], \dots, A[N]$ 。 $A[i]$ 表示第 i 个世界元素能够限制的世界元素的编号。

【输出格式】

一个整数，表示最多可以投放的世界元素的数目。

【输入样例1】

```
6
2 3 1 3 6 5
```

【输出样例1】 3

【说明/提示】数据保证， $1 \leq N \leq 10^6$ ， $1 \leq A[i] \leq N$ 。

动态规划进阶应用

进阶应用3：动态规划+基环树

【基环树的处理】 内向基环树，记 u 指向 v ，用数组 p 记录一个点指向的点，即 $p_u = v$ ；再用链式向前星由 v 向 u 建单向边，外向基环树。

内向基环树优点是找环方便，由于点向内指，只需任选一个点一直往其指向的点走，直到重复为止便找到了环；链式向前星建单向边优势dp方便，删去环上一边后，以断点为根，每个节点其连边都是儿子。

设 $dp_{i,0/1}$ 表示在以 i 根的子树内， i 不选 / 选时的最大选点数。 初始化 $dp_{i,0/1} = 0$ 。

- 不选 i ：则其子节点不受限制，即 $dp_{i,0} = \sum_{j \in \text{son}(i)} \max(dp_{j,0}, dp_{j,1})$
- 选 i ：则其子节点至少有一个不选，即 $dp_{i,1} = 1 + \max_{j \in \text{son}(i)} \left\{ dp_{j,0} + \sum_{j \in \text{son}(i), k \neq j} \max(dp_{k,0}, dp_{k,1}) \right\}$

动态规划进阶应用

进阶应用3：动态规划+基环树

$$dp_{i,1} = 1 + \max_{j \in \text{son}(i)} \left\{ dp_{j,0} + \sum_{j \in \text{son}(i), k \neq j} \max(dp_{k,0}, dp_{k,1}) \right\}$$

当点 i 不被选中时，它的所有子节点都可以自由选择，没有限制。

$$dp_{i,0} = \sum_{k \in \text{son}(i)} \max(dp_{k,0}, dp_{k,1})$$

$$dp_{i,1} = 1 + \max_{j \in \text{son}(i)} \left\{ dp_{j,0} + dp_{i,0} - \max(dp_{j,0}, dp_{j,1}) \right\}$$

$$dp_{i,1} = 1 + dp_{i,0} - \min_{j \in \text{son}(i)} \left\{ \max(dp_{j,0}, dp_{j,1}) - dp_{j,0} \right\}$$

答案为： $\max(dp_{rt,0}, dp_{rt,1})$

【细节】 现考虑删去边的影响，原来有一条边是由根指向一个节点，由于被删除，限制关系被删除，现考虑根限制此节点的情况。

再做一遍dp，选定根节点，即这个点已被限制，该点的子节点也不受任何限制，只需将选这个点的 dp 值上

$\min_{j \in \text{son}(i)} \left\{ \max(dp_{j,0}, dp_{j,1}) - dp_{j,0} \right\}$ 即可，最后答案为 $dp_{rt,1}$ ，与上面答案取最大值即可。

动态规划进阶应用

进阶应用3：动态规划+基环树

【题目04】[ZJOI2008]骑士)

【问题描述】 Z国的骑士团是一个很有势力的组织，帮会中汇聚了来自各地的精英。

他们劫富济贫，惩恶扬善，受到社会各界的赞扬。最近发生了一件可怕的事情，邪恶的Y国发动了一场针对Z国的侵略战争。战火绵延五百里，在和平环境中安逸了数百年的Z国又怎能抵挡的住Y国的军队。于是人们把所有的希望都寄托在了骑士团的身上，就像期待有一个真龙天子的降生，带领正义打败邪恶。

骑士团是肯定具有打败邪恶势力的能力的，但是骑士们互相之间往往有一些矛盾。每个骑士都有且仅有一个自己最厌恶的骑士（当然不是他自己），他是绝对不会与自己最厌恶的人一同出征的。战火绵延，人民生灵涂炭，组织起一个骑士军团加入战斗刻不容缓！

国王交给你一个艰巨的任务，从所有的骑士中选出一个骑士军团，使得军团内没有矛盾的两人（不存在一个骑士与他最痛恨的人一同被选入骑士军团的情况），并且，使得这支骑士军团最具有战斗力。为了描述战斗力，我们将骑士按照1至N编号，给每名骑士一个战斗力的估计，一个军团的战斗力为所有骑士的战斗力总和

【输入格式】 第一行包含一个正整数N，描述骑士团的人数。

接下来N行，每行两个正整数，按顺序描述每一名骑士的战斗力和他最痛恨的骑士。

【输出格式】 应包含一行，包含一个整数，表示你所选出的骑士军团的战斗力。

【输入输出样例】

【输入格式】	【输入格式】
3	30
10 2	
20 3	
30 1	

动态规划进阶应用

进阶应用3：动态规划+基环树

【题目04】 [ZJOI2008]骑士

$$f[i][0] = \sum \{f[son(i)][0], f[son(i)][1]\}$$

$$f[i][1] = \sum \{f[son(i)][0]\}$$

现在有一条环边，dfs找到环边

从该边的两个端点出发选择

(1) 强制不选U, V任意，环的贡献为以U做的f[U][0]

(2) 强制不选V, U任意，环的贡献为以V做的f[V][0]

// 遍历森林

```
void dfs (int x,int father){
    vis[x]=true;
    for(int i=head[x];i!=-1;i=edge[i].nxt){
        int v=edge[i].to;
        if(v==father)continue;
        if(vis[v]){//在一个子树上找到环边
            U=rt,V=v;
            continue;
        }//遍历完整整个子树
        dfs(v,rt);
    }
}
```

```
void tree_dp(int rt){
    dp[rt][1]=val[rt];
    dp[rt][0]=0;
    visdp[rt]=true;
    for(int i=head[rt];i!=-1;i=edge[i].nxt){
        int v=edge[i].to;
        if(visdp[v]) continue;
        tree_dp(v);
        dp[rt][0]+=max(dp[v][0],dp[v][1]);
        dp[rt][1]+=dp[v][0];
    }
}
```

动态规划进阶应用

进阶应用3：动态规划+基环树

【总结】 所有基环树 DP 第一步必须提取环上所有节点，了解判定环的思路（DFS），接下来分治思想 将树部分、环部分分开处理再合并。做题标准化流程为判图类型、提取环、子树预处理、破坏转线性 DP、合并统计全局解。

拓扑排序找环： 将所有度为1的点放入队列，每次取出队首的点并将所有与该点连接的点的度减。只有在环上的点从始至终不会入队，最后未入队的点也就是环上的点。

推荐题目

序号	题目	备注
1	城市环路	P1453
2	[ROIR 2019] 探险队 (Day 2)	P11501
3	[IOI 2008] Island	P4381

动态规划优化（基础）

决策点

决策点：对于选择类的转移方程，比如求最大值、最小值等，由某个点转移到当前状态的，造成转移的称为**决策点**，比如最长上升子序列中：

$$dp_i = \max_{a[i] \geq a[j]} \{dp_j + 1\}$$

转移过程中选到的那个最大的 dp_j ，对应的j就称为**决策点**。

时间戳思想

时间戳思想是很多优化的基石。

考虑dp的**无后效性**，在处理 dp_i 时，一定得到了 $dp_1, dp_2, \dots, dp_{i-1}$ 的所有值：如果我们的决策点是j，那一定**天然满足** $j < i$ ，这里的ij称其为**时间戳**

动态规划优化（基础）

优化1：DP+前缀和优化

【题目06】M – Candies (AtCoder DP Contest M)

【问题描述】 有 N 个孩子，编号为 $1, 2, \dots, N$ 他们决定分享 K 颗糖果。在这里，每个 $i (1 \leq i \leq N)$ ， i 个孩子得到的糖果数 x 必须满足： $0 \leq x \leq a_i$ 。此外，糖果不能剩下。

求他们分享糖果的方法数，由于方案数可能很大，所以结果对 $10^9 + 7$ 取模。

如果有一个孩子分到的糖果数量不同，那么这两种分法就是不同的。

【输入格式】 输入两行，第一行为 N 和 K ；第二行 N 个数表示， $a_1, a_2, \dots, a_N$ 。

【输出格式】 一个数，表示分享糖果的方法数，对 $10^9 + 7$ 取模。

【数据范围】 $1 \leq N \leq 10$ ， $0 \leq K \leq 10^5$ ， $0 \leq a_i \leq K$ 。

【输入样例1】 **【输出样例1】** **【样例1说明】** 以下表示五种分 **【输入样例2】** **【输出样例2】**

3 4	5	配方案：(0,1,3)，(0,2,2)，	2 0	1
1 2 3		(1,0,3)，(1,1,2)，(1,2,1)。	0 0	

动态规划优化（基础）

优化1：DP+前缀和优化

【题目06】M – Candies (AtCoder DP Contest M)

【分析】设 $dp_{i,j}$ 表示前 i 个孩子分 j 颗糖果的方案数。则有转移方程：

$$dp_{i,j} = \sum_{p=j-a_i}^j dp_{i-1,p}$$

初始值: $dp[0][0]=1$ ，目标值: $dp[n][K]$

枚举 i ($1 \sim n$) , j ($0 \sim k$) , p ($0 \sim k$), 时间复杂度是 $O(n \cdot k^2)$

```
11 | dp[0][0]=1;
12 | for(int i=1;i<=n;i++){
13 |     for(int j=k;j>=0;j--){
14 |         dp[i][j]=dp[i-1][j]; //第i个孩子拿0颗糖
15 |         for(int p=1;p<=a[i];p++){
16 |             if(j>=p) dp[i][j]=(dp[i][j]+dp[i-1][j-p])%mod;
17 |         }
18 |     }
19 | }
```

前缀和优化

设 $sum[i][j]$ 表示 $\sum_{p=0}^j dp_{i,p}$ 的结果。

$$dp_{i,j} = sum[i-1][j] - sum[i-1][j-a_i-1]$$

更新完所有的 $dp_{i,j}$ ，再更新下 $sum[i][]$ 。

```
14 | dp[1][0]=sum[1][0]=1;
15 | for(int i=1;i<=k;i++){
16 |     dp[1][i]=(i<=a[1]);
17 |     sum[1][i]=dp[1][i]+sum[1][i-1];
18 | }
19 | for(int i=2;i<=n;i++){
20 |     dp[i][0]=sum[i][0]=1;
21 |     for(int j=1;j<=k;j++){
22 |         if(j<=a[i]) dp[i][j]=sum[i-1][j]%mod;
23 |         else dp[i][j]=(sum[i-1][j]-sum[i-1][j-a[i]-1]+mod)%mod;
24 |         sum[i][j]=(sum[i][j-1]+dp[i][j])%mod;
25 |     }
26 | }
27 | cout<<dp[n][k]<<endl;
28 |
```

动态规划优化（基础）

优化1：DP+前缀和优化

【总结】当 DP 转移方程形如： $dp[i] = f(\sum_{j=a}^b dp[j])$

计算 $dp[i]$ 时需要一步求和 $\text{sum}(dp[a..b])$ ，如果用遍历的方式，则转移的时间复杂度为 $O(N)$ 。如果在状态转移的过程中维护一个dp数组的前缀和sum，则这步求和可以用 $\text{sums}[b+1]-\text{sums}[a]$ 直接得到，转移的时间复杂度变为 $O(1)$ 。DP前缀和优化主要是在转移方程里面出现**简单的求和**时使用。

推荐题目

序号	题目	备注
1	[ABC222D] Between Two Arrays	
2	[ABC179D] Leaping Tak	
3	[HAOI2008] 木棍分割	P2511
4	(agc009_e)E - Eternal Average	

动态规划优化（基础）

优化2: DP+单调队列优化

【题目07】琪露诺（洛谷 P1725）

【问题描述】一列格子依次编号为 0 到 N ，琪露诺只能从编号小的格子移动到编号大的格子。而且琪露诺按照一种特殊的方式进行移动，当她在格子 i 时，她只移动到区间 $[i+L, i+R]$ 中的任意一格。

每一个格子都有一个冰冻指数 A_i ，编号为 0 的格子冰冻指数为 0。当琪露诺停留在那一格时就可以得到那一格的冰冻指数 A_i 。琪露诺希望能够在到达对岸时，获取最大的冰冻指数。开始时，琪露诺在编号 0 的格子上，只要她下一步的位置编号大于 N 就算到达对岸。

【输入格式】

第一行三个正整数 N, L, R 。第二行共 $N+1$ 个整数，第 i 个数表示编号为 $i-1$ 的格子的冰冻指数 A_{i-1} 。

【输出格式】

一个整数，表示最大冰冻指数。

【样例输入1】

5 2 3
0 12 3 11 7 -2

【样例输出1】

11

【数据范围】

对于 60% 的数据， $N \leq 10^4$ 。

对于 100% 的数据， $N \leq 2 \times 10^5$ ， $-10^3 \leq A_i \leq 10^3$ ， $1 \leq L \leq R \leq N$ 。

数据保证最终答案不超过 $2^{31}-1$ 。

动态规划优化（基础）

优化2：DP+单调队列优化

【题目02】琪露诺（洛谷 P1725）

【分析】设 $dp[i]$ 表示到达第 i 个格子能获得的最大权值之和，则转移方程：

$$dp_i = a_i + \max_{i-R \leq j \leq i-L} dp_j$$

时间复杂度： $O(N^2)$

可以考虑直接转化为 RMQ 问题，尝试快速求出这个区间的最大值。

区间长度是固定的 $R-L+1$ ，所以对于**固定区间长度的 RMQ 问题**，可以直接用单调队列快速实现，时间复杂度： $O(N)$ 。

- 
1. 转移到**位置 i** 的位置，为区间 $[i-R, i+L]$ 中， $f[]$ 最大的位置。
 2. 转移到**位置 $i+1$** 的位置，为区间 $[i-R+1, i-L+1]$ 中， $f[]$ 最大的位置。
 3. 转移到**位置 $i+2$** 的位置，为区间 $[i-R+2, i-L+2]$ 中， $f[]$ 最大的位置。



滑动窗口型最值问题

算
法
思
想

顺序枚举 $[L, N]$ 的每一个位置 i 。

1. 将能够转移到 i 的最靠右的位置插入单调队列中；
2. 删除单调队列首不能转移到 i 的位置；
3. 查询当前单调队列首的位置，即为能够转移到 i 的价值最大的位置；
4. 若 $i+R > N$ ，说明位置 i 能够跳到对岸，对此类位置的权值和取最大值，即为答案。

动态规划优化（基础）

优化2：DP+单调队列优化

【题目07】琪露诺（洛谷 P1725）

【总结】：单调队列有两个操作：

- **插入**，新元素从队尾插入后不会破坏单调性的位置为止，再将其插入单调队列；
- **获取最大（小）值**，访问队首元素。

动态规划中的单调队列应用往往牵涉**定长连续区间的最值问题**。

利用单调队列优化动态规划时，每个元素一般存储的两个值：**它在原数列中的位置**（即下标），它在**动态规划中的状态值**。单调队列保证这两个值同时单调。

```
15 dp[0]=0;
16 for(int i=1;i<=n;i++){
17     while (!p.empty()&&dp[p.back()]<=dp[i-1])
18         p.pop_back();
19     while (!p.empty()&&p.front()<i-r)
20         p.pop_front();
21     p.push_back(i-1);
22     dp[i]=dp[p.front()+a[i];
23     if(i+r>n)
24         ans=max(ans,dp[i]);
25 }
```


动态规划优化（基础）

优化2：DP+单调队列优化

【题目08】「ACOI2020」课后期末考试滑溜滑溜补习班（洛谷 P6040）

【问题描述】 有 n 个学生排成一列在问问题，每个人的问题都有一个困难值 a_i ，老师会依次解决同学们的问题。老师解决一个困难值为 a_i 的问题需要花费 a_i 点精力，解决完一个同学的问题，到下一个同学处要花费 k 点精力。特别的，老师也可以不回答下一个同学的问题，直接到下两个、下三个……，假设他解决了 p 号同学的问题，然后直接去解决 q 号同学的问题，则他要花费 $k + (q - p - 1) \times d$ 点精力（ k, d 是给定的参数）

当然，老师不能一直不回答同学的问题，所以他每次最多跳过 $x-1$ 位同学，去解决下 x 位同学的问题。（ x 是给定的参数）问：老师最少花费多少点精力？

【输入格式】 第一行五个整数 n, k, d, x, tp ，表示有 n 个学生，只按顺序去到下一个学生的座位需要花费 k 点精力，每多跳过一个学生就要多花费 d 点精力值，每一次最多只能跳过 $x-1$ 个学生，是否是特殊数据。

- $tp=0$ ，第二行 n 个整数 $a_1 \dots a_n$ ， a_i 表示第 i 个学生的问题的困难值为 a_i 。
- $tp=1$ ，第二行一个整数 **Seed**，作为种子，然后调用 `rnd` 函数依次生成 n 个整数，作为 a 数组， a_i 表示第 i 个学生的问题的困难值为 a_i 。

【输出格式】 一行一个整数，表示杀老师解决完最后一个同学的问题最少需要花费多少精力。

【样例输入1】 5 3 4 1 0 **【样例输出1】** 27
1 2 3 4 5

动态规划优化（基础）

优化2：DP+单调队列优化

【题目03】「ACO12020」课后期末考试滑溜滑溜补习班（P6040）

设 dp_i 表示老师解决完第 i 名同学的问题，可以得到：

$$dp_i = a_i + k + \min_{i-x \leq j \leq i-1} \{dp_j + d \times (i - j - 1)\}$$

初始化 $dp_1 = a_1$ ， 目标解： dp_n

时间复杂度： $O(n^2)$

考虑式子：因为只需要**最优决策点 j** ，相当于固定了 i 去找 j ，所以把 i ， j 分离开，把 i 相关全部看成常数则可以从 $\min$ 里面提出来，得到：

$$dp_i = a_i + k + d \times (i - 1) +$$

$$\min_{i-x \leq j \leq i-1} \{dp_j - d \times j\}$$

所以我们只需要实现一个维护 $dp_j - d \times j$ 最小值的单调队列。

时间复杂度： $O(n)$

```
22 dp[1]=a[1];
23 q.push_back(1);
24 for(int i=2;i<=n;i++){
25
26     while(!q.empty()&&i-q.front()>x)
27         q.pop_front();
28
29     dp[i]=dp[q.front()]+a[i]+k+(i-q.front()-1)*d;
30
31     while(!q.empty()&&dp[i]-i*d<=dp[q.back()]-q.back()*d)
32         q.pop_back();
33
34     q.push_back(i);
35 }
```


动态规划优化（基础）

优化2：DP+单调队列优化

【题目09】[SCOI2010]股票交易（P2569）

【题目描述】

最近lxhgww又迷上了投资股票，通过一段时间的观察和学习，他总结出了股票行情的一些规律。通过一段时间的观察，lxhgww预测到了未来 T 天内某只股票的走势，第 i 天的股票买入价为每股 AP_i ，第 i 天的股票卖出价为每股 BP_i （数据保证对于每个 i ，都有 $AP_i \geq BP_i$ ），但是每天不能无限制地交易，于是股票交易所规定第 i 天的一次买入至多只能购买 AS_i 股，一次卖出至多只能卖出 BS_i 股。

另外，股票交易所还制定了两个规定。为了避免大家疯狂交易，股票交易所规定在两次交易（某一天的买入或者卖出均算是一次交易）之间，至少要间隔 W 天，也就是说如果在第 i 天发生了交易，那么从第 $i+1$ 天到第 $i+W$ 天，均不能发生交易。同时，为了避免垄断，股票交易所还规定在任何时间，一个人的手里的股票数不能超过 $MaxP$ 。

在第1天之前，lxhgww手里有一大笔钱（可以认为钱的数目无限），但是没有任何股票，当然， T 天以后，lxhgww想要赚到最多的钱，聪明的程序员们，你们能帮助他吗？

【输入格式】 输入数据第一行包括3个整数，分别是 T ， $MaxP$ ， W 。

接下来 T 行，第 i 行代表第 $i-1$ 天的股票走势，每行4个整数，分别表示 AP_i ， BP_i ， AS_i ， BS_i 。

【输出格式】

输出数据为一行，包括1个数字，表示lxhgww能赚到的最多的钱数。

对于 30%的数据， $0 \leq W < T \leq 50, 1 \leq MaxP \leq 50$ ；
对于 50% 的数据， $0 \leq W < T \leq 2000, 1 \leq MaxP \leq 50$ ；
对于 100% 的数据， $0 \leq W < T \leq 2000, 1 \leq MaxP \leq 2000$ ；
对于所有的数据， $1 \leq BP_i \leq AP_i \leq 1000, 1 \leq AS_i, BS_i \leq MaxP$ 。

【输入样例1】

```
5 2 0
2 1 1 1
2 1 1 1
3 2 1 1
4 3 1 1
5 4 1 1
```

【输出样例2】3

动态规划优化（基础）

优化2：DP+单调队列优化

【题目09】 [SCOI2010]股票交易（P2569）

【分析】设 $dp[i][j]$ 第 i 天结束，手中持有 j 股时的最大盈利。初始 $dp[0][0]=0$ ，其余为负无穷；答案： $\max_{0 \leq j \leq MaxP} dp[T][j]$

分类讨论当天的四个选择：

- 不操作：继承前一天的持股状态

$$dp[i][j] = \max\{dp[i][j], dp[i-1][j]\}$$

- 买入操作：直接买 j 股，支出操作，利润为负。

$$dp[i][j] = \max\{dp[i][j], -j \times AP_i\} \quad 0 \leq j \leq AS_i$$

- 加购至 j 股：上次交易最早只能在 $i-W-1$ 天，之前持有 k 股，买入 $j-k$ 股，最多买入 AS_i ： $j - AS_i \leq k \leq j$

$$dp[i][j] = \max\{dp[i-W-1][k] + k \times AP_i\} - j \times AP_i$$

- 卖出减少至 j 股：最早只能在 $i-W-1$ 天，现在卖出 $k-j$ 股，单次最多卖 BS_i ： $j \leq k \leq j + BS_i$

$$dp[i][j] = \max\{dp[i-W-1][k] + k \times BP_i\} - j \times BP_i$$

动态规划优化（基础）

优化2：DP+单调队列优化

【总结】DP转移区间无固定左右边界,而且代价同时依赖*i*和*j*，这种情况是没法使用单调队列优化的。常见解决方式为，首先完成DP方程的书写，再此基础上进行等式转换，化简为滑动窗口问题求最值，再进一步使用单调队列维护。书写过程中需要注意边界和存储的内容。

推荐题目

序号	题目	备注
1	[USACO19JAN] Redistricting P	P5202
2	[蓝桥杯 2025 国 C] 数列染色	P12882
3	『STA - R4』踱步	P10119
4	Fence	P10978

动态规划优化（基础）

优化3：DP+线段树优化

静态的数据结构维护的条件限制一般较为简单，那如果考虑复杂的限制条件，或者要维护动态变化的RMQ等问题，就需要使用线段树。一般是用于快速查询决策点，把 $O(N^2)$ 优化到 $O(N \log N)$ 。

【题目10】 Flowers (AT_dp_q)

有 N 朵花排成一排。对于每个 $i(1 \leq i \leq N)$ ，从左往右第 i 朵花的高度为 h_i ，美丽值为 a_i 。其中 $h_1, h_2, \dots, h_n$ 互不相同。

太郎君想通过拔掉一些花，使得剩下的花满足以下条件：从左到右看，剩下的花的高度严格递增。请你求出剩下的花的美丽值总和的最大值。

【输入格式】 输入 $N, h_1, h_2, \dots, h_n; a_1, a_2, \dots, a_n$ 。

【输出格式】 请输出剩下的花的美丽值总和的最大值。

【样例输入1】 【样例输出1】 【样例说明】 保留从左到右第 2、4 朵

4 60 花即可。此时高度依次为 1,2，满足严

3 1 4 2 格递增。美丽值总和为 $20+40=60$ 。

10 20 30 40

【数据范围】 $1 \leq N \leq 2 \times 10^5, 1 \leq h_i \leq 2 \times 10^5, h_1, h_2, \dots, h_n$ 互不相同, $1 \leq a_i \leq 10^9$ 。

动态规划优化（基础）

优化3：DP+线段树优化

【题目10】 Flowers (AT_dp_q)

设 dp_i 表示以第 i 朵花结尾的最大美丽值之和，则有：

$$dp_i = a_i + \max_{h[j] < h[i], j < i} \{dp_j\}$$

【优化】：如何快速找出最大满足 $h[j] < h[i]$ ， $j < i$ 的 dp_j ，**时间戳思想**：因线性dp， $j < i$ 这个条件在循环过程中就已经天然满足了，只需要快速找出满足 $h[j] < h[i]$ 的 dp_j 就可以。

查询使用权值线段树：以 h_i 作为下标， dp_i 作为值来构建线段树，查询的时候查询区间 $[1, h_i - 1]$ 的最大值即可。

正确性：因为时间戳的原因，权值线段树里面的值初始全都是0，而不是0的地方一定是前面遇到过的，所以区间 $[1, h_i - 1]$ 包括：还没刷新的状态（这些状态还对应着0，所以不会影响状态）、满足 $h_j < h_i$ 的 j （这些状态就对应着每一个满足条件的 dp_j ）

线段树操作：

查询区间 $[1, h_i - 1]$ 的最小值 $minn$ 。得到 $dp_i = a_i + minn$ ，在权值线段树的 h_i ，单点修改为 dp_i ，时间复杂度： $O(N \log N)$ 。

动态规划优化（基础）

优化3: DP+线段树优化

【题目11】[TJOI2011] 书架 (P1295)

【题目大意】给出一个长度为 n 的序列 h ，请将 h 分成若干段，满足每段数字之和都不超过 m ，最小化每段的最大值之和。

【数据范围】对于 30% 的数据，保证 $n \leq 10^3$ 。

对于 100% 的数据，保证 $1 \leq n \leq 10^5$ ， $1 \leq h_i \leq 10^9$ ， $\max_{1 \leq i \leq n} h_i \leq m \leq 10^9$ 。

【分析】设 f_i 表示已经分好 i 个数字的最小代价。

$$f_i = \min\{f_{k-1} + \max_{k \leq j \leq i} h_j\} \quad \left(\sum_{j=k}^i h_j \leq m \right)$$

给定 i ，当 k 增加时， $\max_{k \leq j \leq i} h_j$ 单调不增， f_{k-1} 单调不降，平衡点？

思考 h_i 对 $\max_{k \leq j \leq i} h_j$ 的影响？

动态规划优化（基础）

优化3: DP+线段树优化

【题目11】[TJOI2011] 书架 (P1295)

设 i 左侧第一个满足 $h_i > h_j$ 的位置为 p_i , $\max_{k \leq j \leq i} h_j$ ($k > p_i$) 都会变为
线段树维护位置 k 的 f_{k-1} 和 $f_{k-1} + \max_{k \leq j \leq i} h_j$
 h_i 。

【理解】你有一串高度 $[5, 3, 4]$, 现在新来一个 $h_i = 4.5$ 。
它左边第一个比它大的是 5 (位置 p)。从 5 之后
到 i 的这段区间 $([3, 4])$, 原来最大值是 4, 现在因为
新元素 4.5 加入, 这段的最大值都变成了 4.5。

$\max_{k \leq j \leq i} h_j$ 可用支持区间赋值的数据结构进行维护,
转移 f_i 时, 需要进行区间查询。

当枚举到一个新的 h_i 时:

1. 单调修改, 更新位置 $k=i$ 时的 f_{k-1} 。
2. 根据 h_i 更新区间 $[p_i + 1, i]$ 的 $f_{k-1} + \max_{k \leq j \leq i} h_j$ 。
3. 二分得到第一个不满足 $\max_k h_i \leq m$ 的位置 l , 则 $k \in [l + 1, i]$ 。
4. 查询 $[l+1, i]$ 的最小的 $f_{k-1} + \max_{k \leq j \leq i} h_j$

动态规划优化（基础）

优化3：DP+线段树优化

【总结】单调队列优化 DP 仅适用于转移区间是连续滑动窗口，即转移的合法 j 一定是一段连续、随 i 匀速平移的区间 $([i-L, i-R])$ 。而线段树优化 DP 适用范围更广，核心判定条件。DP 转移方程形式 $dp[i] = \min/\max\{dp[j]\} + val(i)$ ，其中 j 的合法取值是任意一段 / 多段静态区间，不满足滑动窗口特性；合法转移下标 j 无统一左右偏移关系，边界不随 i 同步平移；存在值域限制、离散下标、分段不连续、左右边界独立变化、二维单点查询等场景。

推荐题目：

序号	题目	备注
1	选择数字	P2034
2	yyy loves OI IV	P2418
3	[USACO04DEC] Dividing the Path G	P1545

动态规划优化（基础）

优化4: DP+斜率优化

考虑一类状态转移方程：

$$DP_i = \min/\max\{DP_j + A_i B_j + C_i + D_j\}$$

其中 DP_i , A_i , B_i 是和 i 有关的函数, DP_j , B_j , D_j 是和 j 有关的函数,

如果没有 $A_i B_j$,那么原式可以转为: $DP_i - C_i = \min/\max\{DP_j + D_j\}$ 可直接用单调队列, 维护一个最小/最大的 $DP_j + D_j$ 的单调队列即可。因 $A_i B_j$ 无法使用单调队列优化。

【转换】：

$(DP_i - C_i) = \min\{(DP_j + D_j) + A_i B_j\}$, 寻找决策点 j 转移给 i , j 便是里面取得最小值得那个点。

类比一次函数 $y=kx+b \rightarrow \mathbf{b=y-kx}$, 将上述得四个值对应转移方程得四个值:

$$b_i = DP_i - C_i$$

$$y_i = DP_j + D_j$$

$$k_i = -A_i$$

$$x_j = B_j$$

找决策点 j , 而 i 固定, A_i 是固定的, 最小化 DP_i 等价于最小化 b 截距。

动态规划优化 (基础)

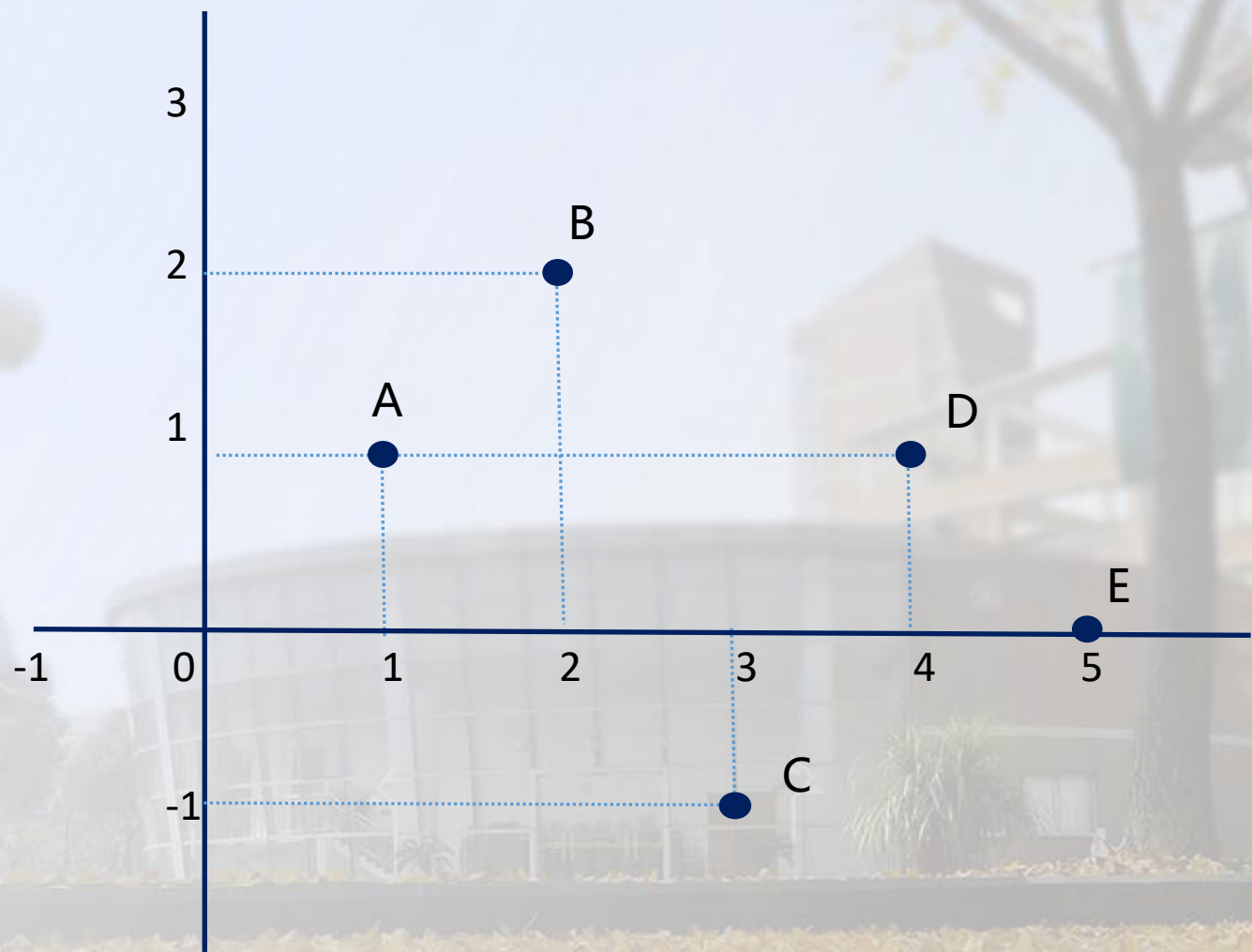
优化4: DP+斜率优化

求解最小截距 $(DP_i - C_i) = \min\{(DP_j + D_j) + A_i B_j\}$

$$b_i = \min\{y_j - k_i x_j\}$$

- 固定一个 i , 那么 k_i 固定, 设 $len=r-l+1$, 对于 $[l, r]$ 之间的所有 j , 对应于一个大小为 len 点集 (x_j, y_j) 。如图五个点且 x_j 严格递增。
- 一个过 (x_j, y_j) 且斜率为 k_i 的直线, 是唯一存在。
- len 个点对应于 len 条直线。

由右图观察, 截距最小的直线CE来进行转台转移。



动态规划优化（基础）

优化4: DP+斜率优化

求解最小截距 $(DP_i - C_i) = \min\{(DP_j + D_j) + A_i B_j\}$

$$b_i = \min\{y_i - k_i x_j\}$$

假设场景: x_j 递增, k_i 递增的情况。

一般的题目中, 这个 $[l, r]$ 区间, 往往取的是 $[1, i-1]$

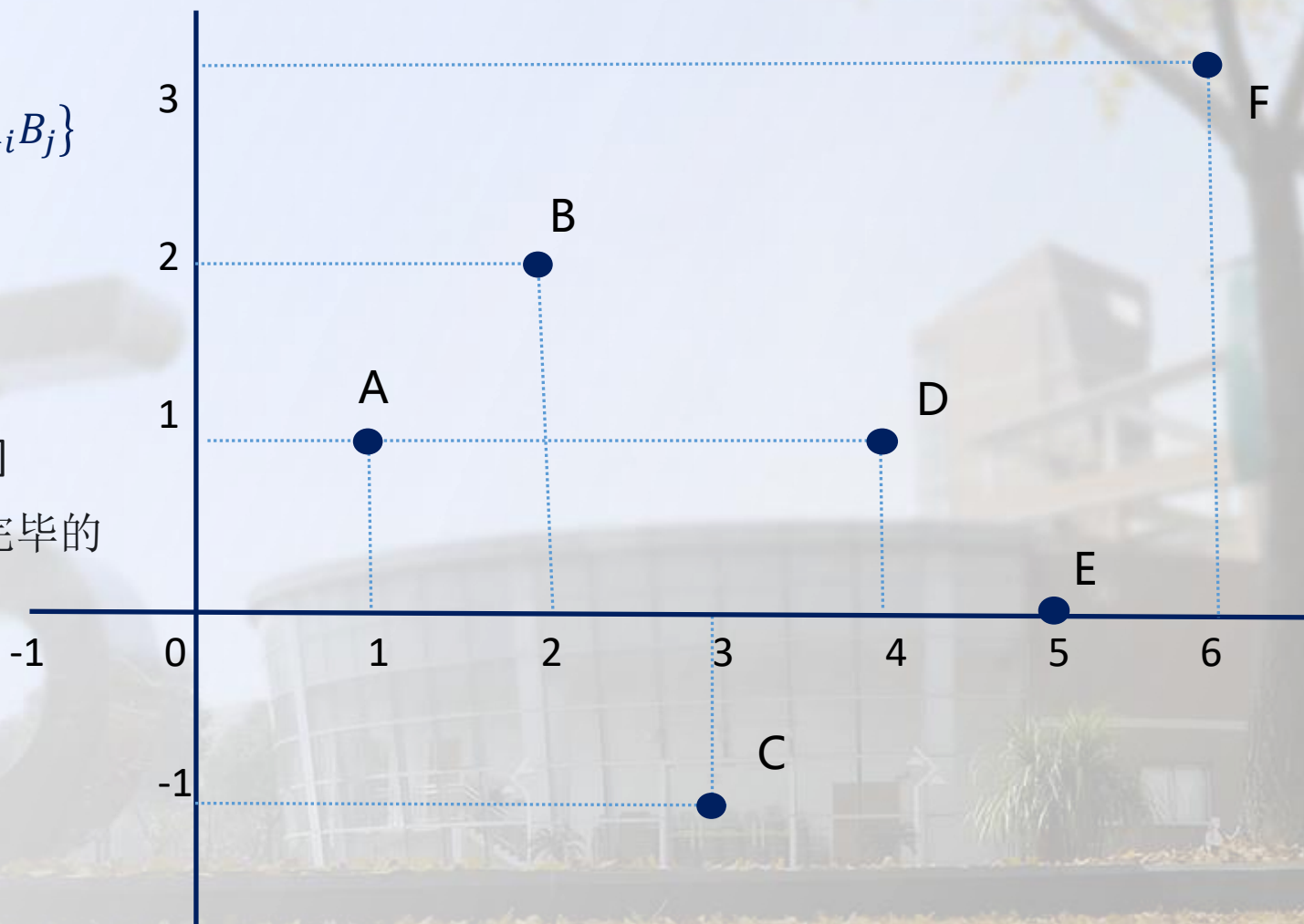
这个点集的大小只会被增加不会减小, 而计算完毕的

$dp[i]$, 又往往和 y_i 有关,

所以我们在计算 $dp[i+1]$ 的时候,

就需要加入 (x_i, y_i) 到这个点集中。

•新加入F点。



动态规划优化（基础）

优化4: DP+斜率优化

求解最小截距 $(DP_i - C_i) = \min\{(DP_j + D_j) + A_i B_j\}$

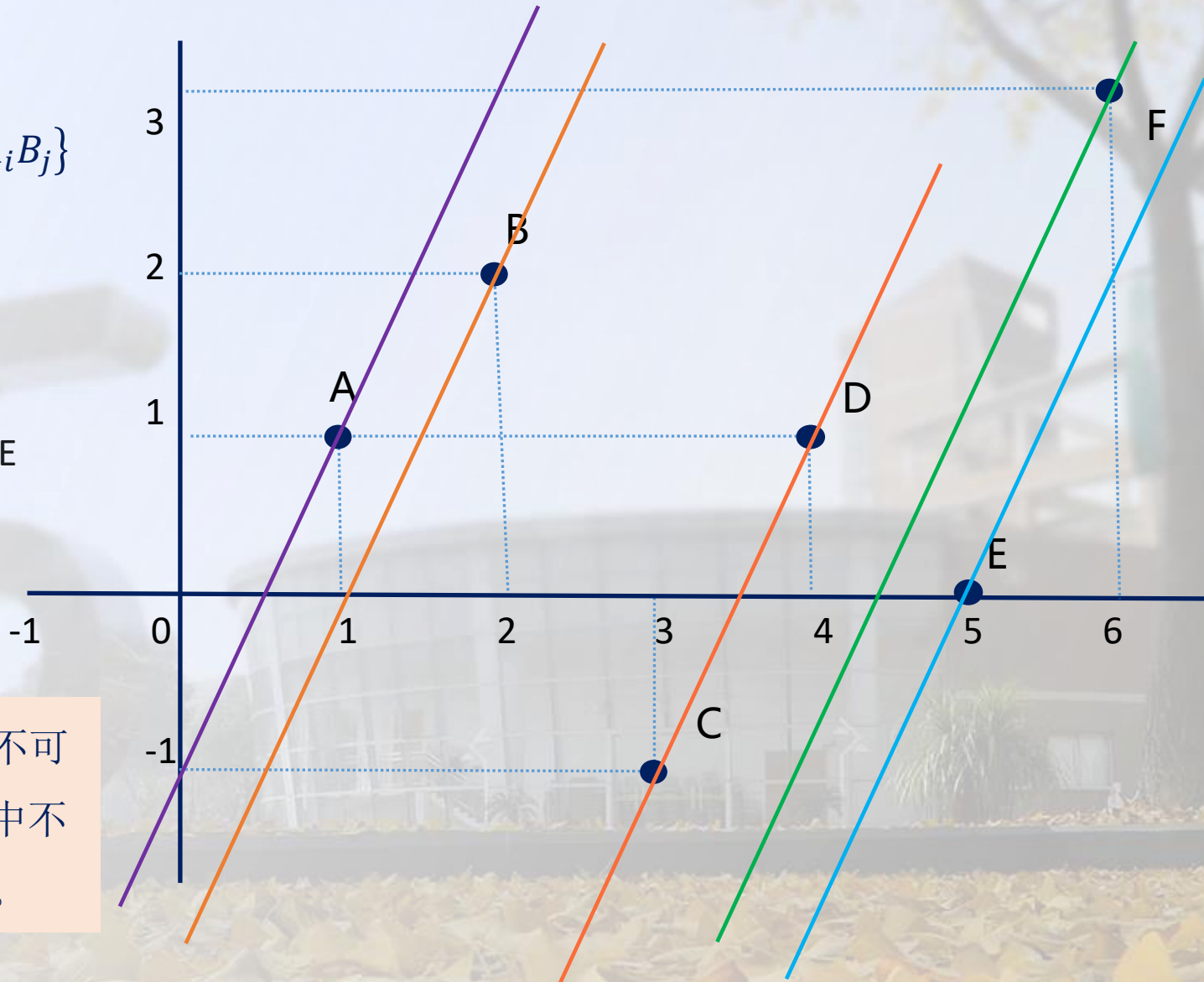
$$b_i = \min\{y_i - k_i x_j\}$$

•新加入F点。

此时斜率 k_i 也增大，假设 $k_i = 2$ ，可以看到此时E点所在直线对应的截距是最小的。

状态转移，转换为“点集”中选取最小截距。

【思考】 分析不论斜率 k_i 值为多少，B、D点都不可能当作转移状态的前驱，如果能够剔除点集中不可能转移的点，计算量减小，复杂度得到优化。



动态规划优化（基础）

优化4: DP+斜率优化

求解最小截距 $(DP_i - C_i) = \min\{(DP_j + D_j) + A_i B_j\}$

$$b_i = \min\{y_i - k_i x_j\}$$

假设有两个点A,B, 设二者所在直线的斜率为 k_{AB} .

如果过AB的直线在旋转, 斜率随之变化, 旋转过程中有时A点优, 有时B点优。

旋转的边界值: $k_i = k_{AB}$;

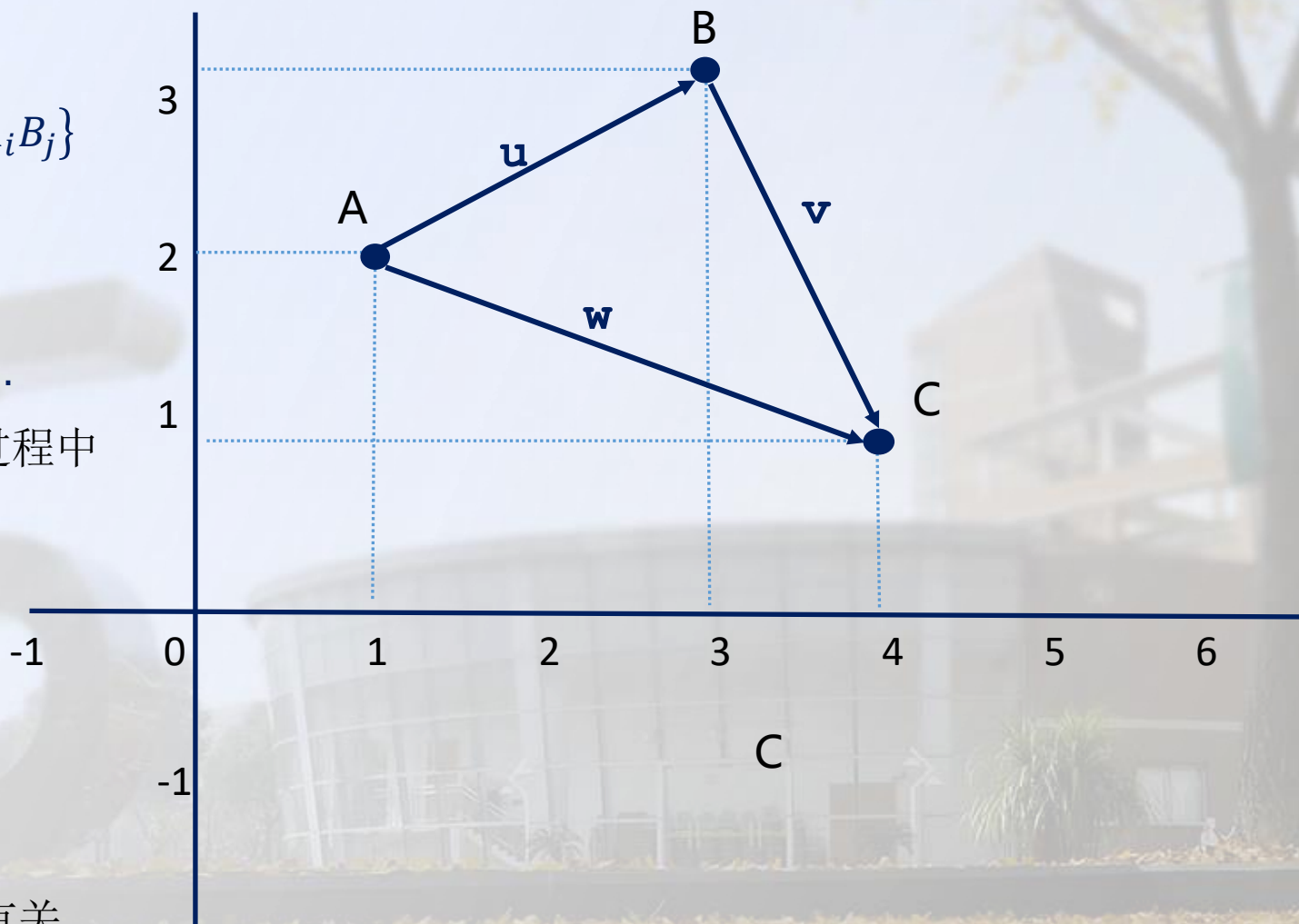
斜率不存在是 (平行y轴)

旋转经过这两个边界时, 最优点发生改变。

对于AC来说, B点时一个上凸点。

总结可得最优点, k_i 与 k_{AB}, k_{BC}, k_{AC} 的大小关系有关。

目前: $k_{BC} < k_{AC} < 0 < k_{AB}$, 经过这三个点的直线在旋转, 斜率随之变化: $k_i \rightarrow -\infty$ 向 $k_i \rightarrow +\infty$, 斜率一直增大。



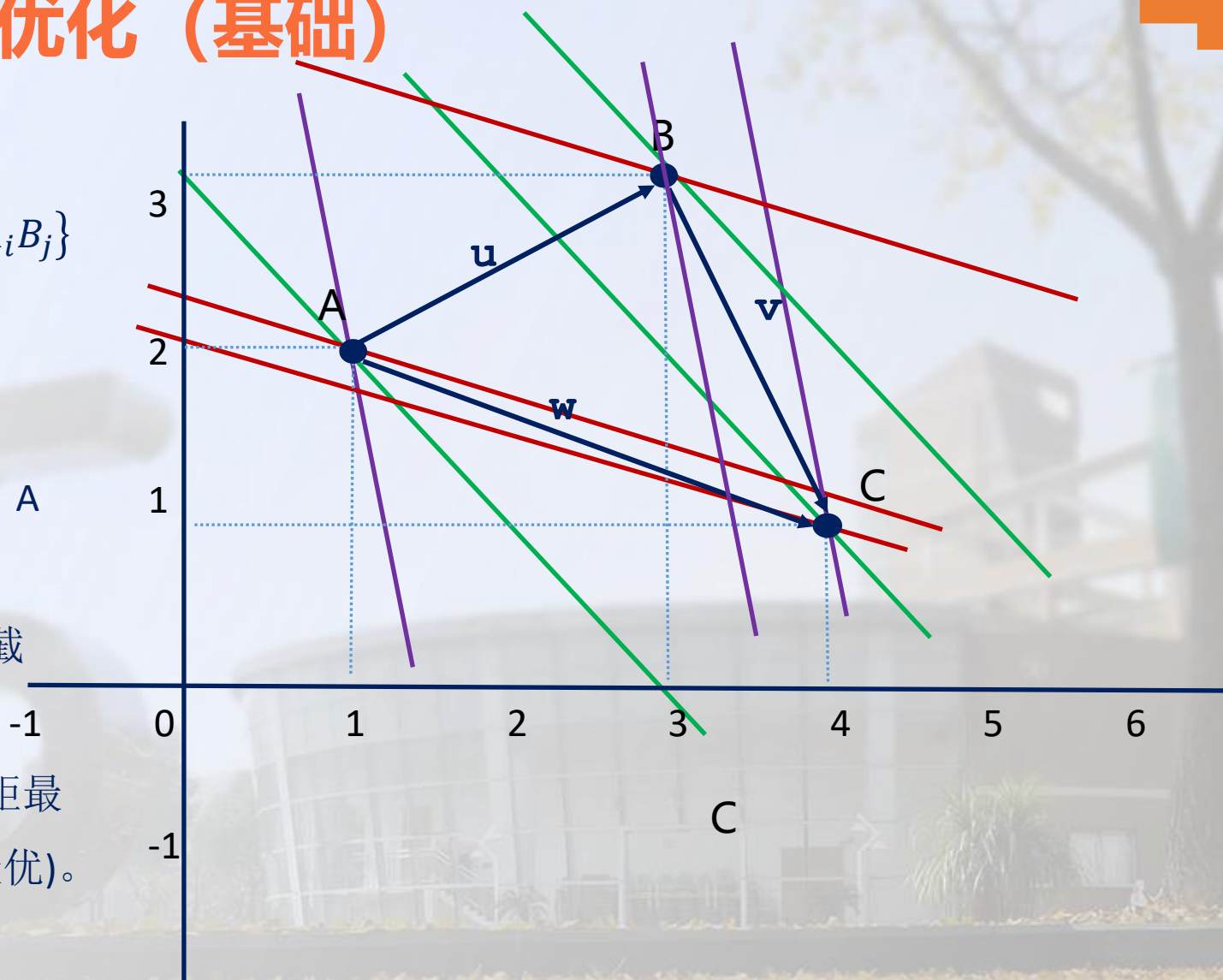
动态规划优化 (基础)

优化4: DP+斜率优化

求解最小截距 $(DP_i - C_i) = \min\{(DP_j + D_j) + A_i B_j\}$

$$b_i = \min\{y_i - k_i x_j\}$$

- 当时 $k_i \leq k_{BC}$, A点所在直线(紫色)截距最小, A点最优。
- 当时 $k_{BC} \leq k_i \leq k_{AC}$, A点所在直线(绿色)截距最小, A点最优。
- 当时 $k_{AC} \leq k_i \leq 0$, C点所在直线(红色)截距最小(划分不同斜率区间, 重合点A,C同时取到最优)。



动态规划优化（基础）

优化4: DP+斜率优化

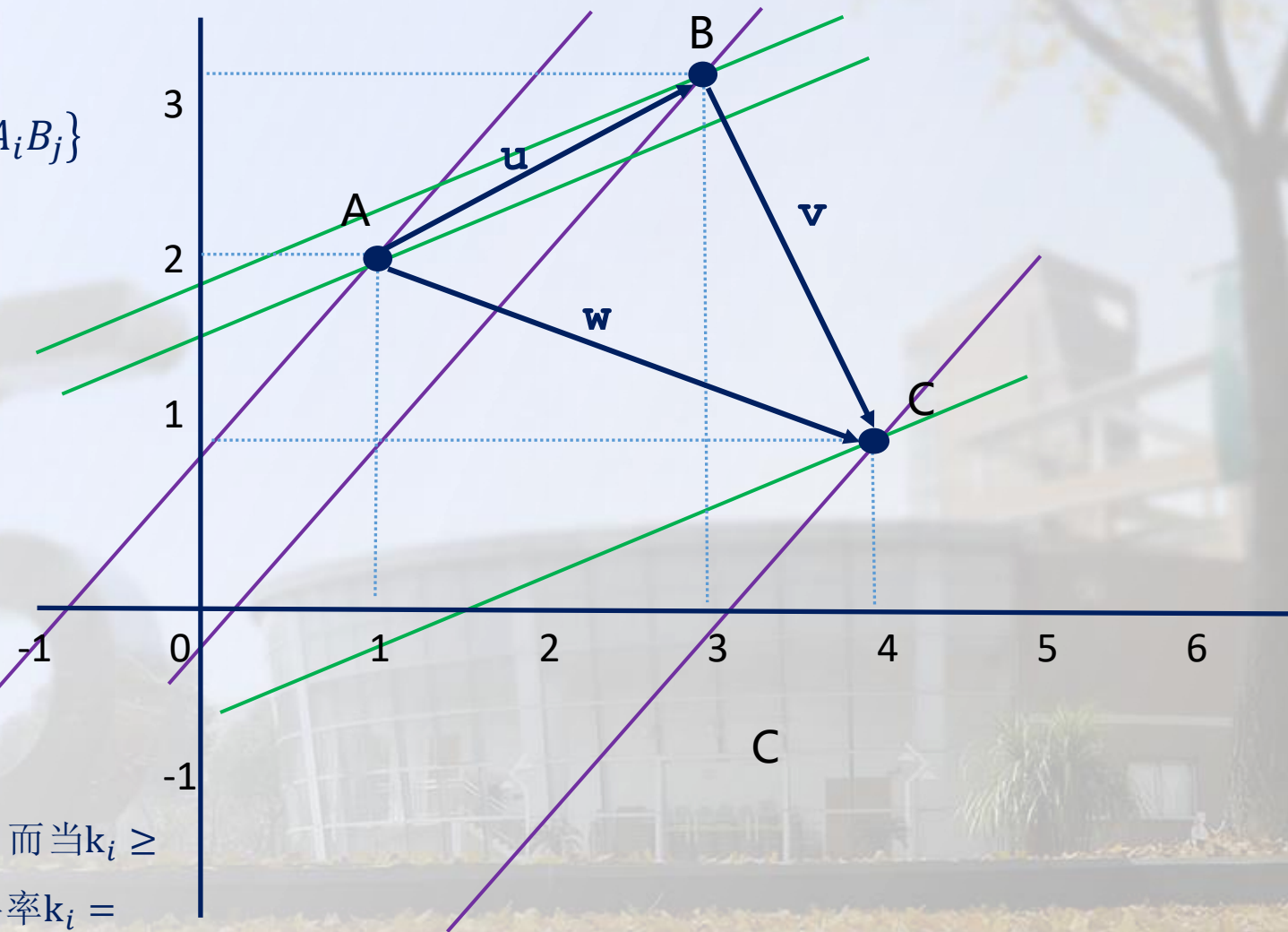
求解最小截距 $(DP_i - C_i) = \min\{(DP_j + D_j) + A_i B_j\}$

$$b_i = \min\{y_i - k_i x_j\}$$

- 当 $0 \leq k_i \leq k_{AB}$, C点依旧是最优点。（绿色）
- 当 $k_i \geq k_{AB}$, C点是最优点。（紫色）

总结: 1. 无论取什么值, 都不可能成为最优点。所以这样的“上凸点”对于状态转移来说, 是冗余点, 舍弃不参与计算。

2. 因为 k_i 是递增, 当 $k_i < k_{AC}$ 时, 最优点一直都是A点。而当 $k_i \geq k_{AC}$ 时, 最优点一直都是C点。考虑两个点的情况, 斜率 $k_i = k_{AC}$ 就是最优点的分界线。



动态规划优化（基础）

优化4: DP+斜率优化

求解最小截距 $(DP_i - C_i) = \min\{(DP_j + D_j) + A_i B_j\}$

$$b_i = \min\{y_i - k_i x_j\}$$

【思路】 维护一个下凸包就可以减少大量的无用计算，在凸包上找到一个点 P_i ，其前后的点分别为 P_{i-1} ， P_{i+1} 满足 $k_{P_{i-1}P_i} \leq k_i \leq k_{P_iP_{i+1}}$ 。这样的话，因为 $k_{P_{i-1}P_i} \leq k_i$ ，所以 P_i 比它前面的点都优秀，又，所以又 $k_i \leq k_{P_iP_{i+1}}$ 后面的点都优秀，综述，它就是全局最优点，也就是我们状态转移的前驱。

因为 k_i 是递增，二分查找最优点，其一直右移具有单调性。可使用单调队列，当头部满足 $k < k_i$ 时候就出队，时刻维护头部的 $k > k_i$ 。每次取到的都是最优点。另外新加入的点，也就相当于放入单调队列的尾部，尾部进行去除上凸点的操作。

动态规划优化（基础）

【题目05】任务安排（P2365）

【题目描述】

n 个任务排成一个序列在一台机器上等待完成（顺序不得改变），这 n 个任务被分成若干批，每批包含相邻的若干任务。

从零时刻开始，这些任务被分批加工，第 i 个任务单独完成所需的时间为 t_i 。在每批任务开始前，机器需要启动时间，而完成这批任务所需的时间是各个任务需要时间的总和（同一批任务将在同一时刻完成）。

每个任务的费用是它的完成时刻乘以一个费用系数 f_i 。请确定一个分组方案，使得总费用最小。

【输入格式】

第一行一个正整数 n 。

第二行是一个整数 s 。

下面 n 行每行有一对数，分别为 t_i 和 f_i ，表示第 i 个任务单独完成所需的时间是 t_i 及其费用系数 f_i 。

【输出格式】

一个数，最小的总费用。

【样例输入 #1】 【样例输出 #1】

对于 100% 的数据， $1 \leq n \leq 5000$ ， $0 \leq s \leq 50$ ， $1 \leq t_i, f_i \leq 100$ 。

5
1
153

【样例解释】如果分组方案是：

{1, 2}, {3}, {4, 5}, 则完成时间分别为{5, 5, 10,
14, 14}, 费用 $C=15+10+30+42+56$,总费用就是153。

1 3
3 2
4 3
2 3
1 4

动态规划进阶应用

【题目05】任务安排（P2365）

【分析】设 $dp[i][j]$ 表示前 i 个任务分成 j 组完成的最小代价。

$1, 2, 3, \dots, k-1, k, k+1, \dots, i$

$$dp[i][j] = \min_{1 \leq k \leq i} \{ dp[k][j-1] + (S \times j + \sum_{x=1}^i T_x) \times \sum_{x=k+1}^i f_x \}$$

$$\text{令 } \text{sum}T_i = \sum_{k=1}^i T_k, \quad \text{sum}C_i = \sum_{k=1}^i f_k$$

$$dp[i][j] = \min_{1 \leq k \leq i} \{ dp[k][j-1] + (S \times j + \text{sum}T_i) \times (\text{sum}C_i - \text{sum}C_k) \}$$

答案: $\sum_{j=1}^n \min\{dp[n][j]\}$ 时间复杂度: $O(n^3)$

```
15 | for(int i=1;i<=n;i++)
16 |     dp[i][1]=(sumT[i]+s)*sumC[i]; //初始化
17 | for(int i=2;i<=n;i++)
18 |     for(int j=2;j<=i;j++)
19 |         for(int k=1;k<i;k++)
20 |             dp[i][j]=min(dp[i][j],dp[k][j-1]+(s*j+sumT[i])*(sumC[i]-sumC[k]));
```

思考：根据数据范围分析会出现超时和超内存，需要进一步**优化**。

动态规划进阶应用

【题目05】任务安排（P2365）

【优化】 $dp[i][j]$ 中 j 只与以后任务生产的代价有关，**降维**。

$1, 2, 3, \dots, k-1, k, k+1, \dots, i$

设 $dp[i]$ 表示前 i 个任务的最小费用总和。

依然考虑将任务枚举前 k 个， $[k+1, i]$ 分为一批。前 k 个任务完成最小代价 $dp[k]$ 。

计算代价分为两部分： $\sum_{j=1}^i T_j \times \sum_{j=k+1}^i p_j$ 和 $S \times \sum_{j=k+1}^i p_j$ 。所以可得转移方程：

$$dp[i] = \min_{0 \leq k < i} \{ dp[k] + \sum_{j=1}^i T_j \times \sum_{j=k+1}^i p_j + S \times \sum_{j=k+1}^i p_j \}$$

$$\text{令 } \text{sum}T_i = \sum_{k=1}^i T_k, \quad \text{sum}C_i = \sum_{k=1}^i f_k$$

$$dp[i] = \min_{0 \leq k < i} \{ dp[k] + S \times (\text{sum}C_n - \text{sum}C_k) + \text{sum}T_i \times (\text{sum}C_i - \text{sum}C_k) \}$$

初始值： $dp[0]=0$ 答案： $dp[n]$ 时间复杂度： $O(n^2)$

```
16 dp[0]=0;
17 for(int i=1;i<=n;i++)
18     for(int k=0;k<i;k++)
19         dp[i]=min(dp[i],dp[k]+S*(sumC[n]-sumC[k])+sumT[i]*(sumC[i]-sumC[k]));
20 cout<<dp[n]<<"\n";
```


动态规划进阶应用

【题目06】任务安排2 (P10979, P2365 强化版)

数据范围：对于 100% 的数据， $1 \leq n \leq 3 \times 10^5$ ， $0 \leq s \leq 2^8$ ， $1 \leq t_i \leq 2^8$ ， $0 \leq f_i \leq 2^8$ 。

$$dp[i] = \min_{0 \leq k < i} \{dp[k] + S \times (\text{sum}C_n - \text{sum}C_k) + \text{sum}T_i \times (\text{sum}C_i - \text{sum}C_k)\}$$

$$dp[i] = dp[k] + S \times (\text{sum}C_n - \text{sum}C_k) + \text{sum}T_i \times (\text{sum}C_i - \text{sum}C_k)$$

$$dp[i] = dp[k] + S \times \text{sum}C_n - S \times \text{sum}C_k + \text{sum}T_i \times \text{sum}C_i - \text{sum}T_i \times \text{sum}C_k$$

$$y = x \cdot k + b$$

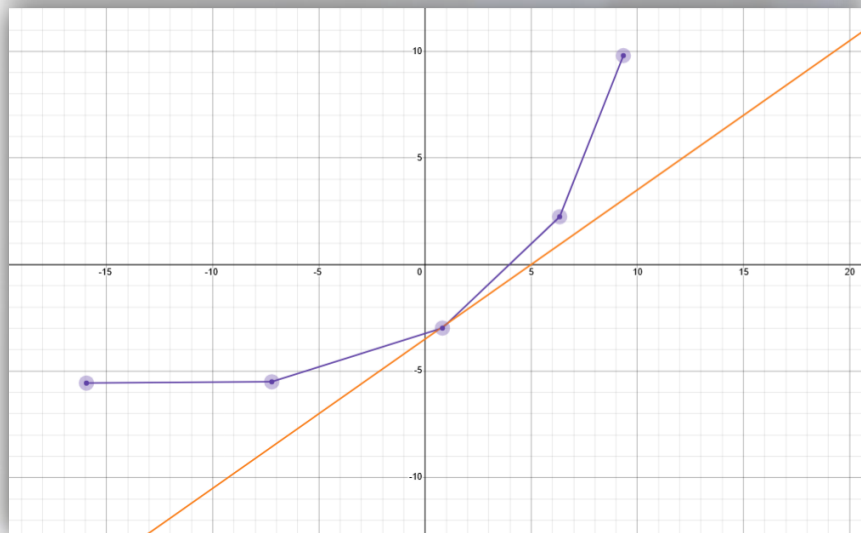
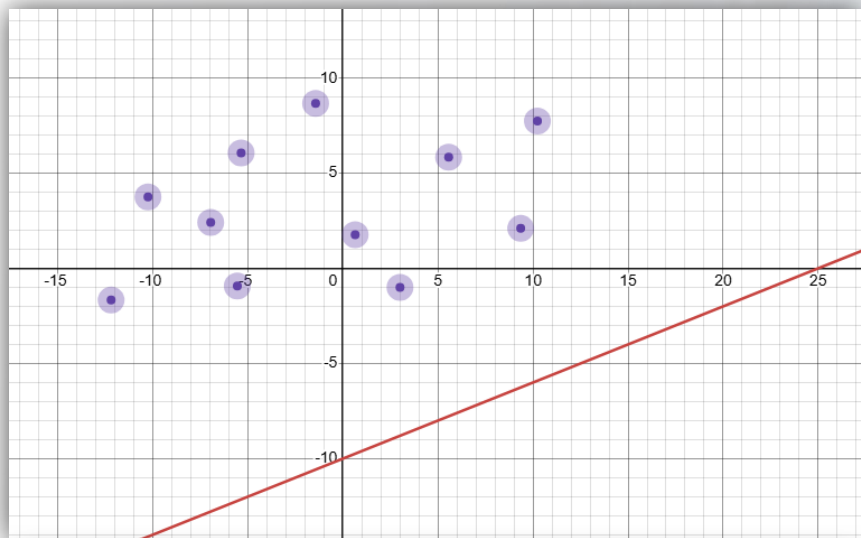
斜率优化

$$dp[k] = \text{sum}C_k (S + \text{sum}T_i) + dp[i] - S \times \text{sum}C_n + \text{sum}T_i \times \text{sum}C_i$$

横坐标 $\text{sum}C_k$ ，纵坐标 $dp[k]$ ，斜率为： $S + \text{sum}T_i$ ，截距为： $dp[i] - S \times \text{sum}C_n + \text{sum}T_i \times \text{sum}C_i$

动态规划进阶应用

【题目06】任务安排2（P10979, P2365 强化版）



$$dp[k] = \text{sum}C_k(S + \text{sum}T_i) + dp[i] - S \times \text{sum}C_n + \text{sum}T_i \times \text{sum}C_i$$

$$y = x \cdot k + b$$

当*i*固定时，直线的斜率也是固定的，每当直线经过一点可以得到一个截距。
当截距最小时， $dp[i]$ 取到最小值。

设1, 2两点的连线的斜率 k_1 ，2, 3两点的连线斜率为 k_2 ，进行决策的直线斜率为 K ，一个点是最优解，则 $k_1 \leq K \leq k_2$ 。

😞 由于 $0 \leq k < i$ ，当*i*增加时会有新的点加入， $\text{sum}C_i$ 具有单调性。根据本题的情况只有下凸壳中的点才能可能最优，下凸壳中线段斜率单调递增，可用单调队列维护。

$\text{sum}T_i$ 是单调递增（斜率），凸包最右边的未选上的点不可能被选上，所以将其弹出队列，队列最右端的点即为答案。

动态规划进阶应用

【题目06】任务安排2 (P10979, P2365 强化版)

核心代码实现

```
14  memset(dp, 0x3f, sizeof dp);
15  dp[0]=0; l=r=1;
16  q[1]=dp[0];
17  for(int i=1; i<=n; i++){
18      while(l<r&&(dp[q[l+1]]-dp[q[l]])<=(s+sumT[i])*(sumC[q[l+1]]-sumC[q[l]])) l++;
19      // 最右侧两点形成线段的斜率小于当前决策斜率, 右侧点不可能再被使用, 弹出队列。
20      int k=q[l];
21      dp[i]=dp[k]+s*(sumC[n]-sumC[k])+sumT[i]*(sumC[i]-sumC[k]);
22      while(l<r&&(dp[q[r]]-dp[q[r-1]])*(sumC[i]-sumC[q[r]])>=(dp[i]-dp[q[r]])*(sumC[q[r]]-sumC[q[r-1]])) r--;
23      // 维护单调队列中斜率的单调性
24      q[++r]=i;
25  }
```

【题目07】[SDOI2012] 任务安排 (P5785)

数据范围：对于 100% 的数据, $1 \leq n \leq 3 \times 10^5$, $0 \leq s \leq 2^8$, $1 \leq t_i \leq 2^8$, $0 \leq f_i \leq 2^8$ 。



数据范围：对于 100% 的数据, $1 \leq n \leq 3 \times 10^5$, $0 \leq s \leq 2^8$, $1 \leq |T_i| \leq 2^8$, $0 \leq C_i \leq 2^8$ 。

动态规划进阶应用

【题目07】 [SDOI2012] 任务安排 (P5785)

$$y = x \cdot k + b$$

$$dp[k] = \text{sum}C_k (S + \text{sum}T_i) + dp[i] - S \times \text{sum}C_n + \text{sum}T_i \times \text{sum}C_i$$

分析：【题目06】到【题目07】变换在于 $1 \leq |T_i| \leq 2^8$ 。 $\text{sum}T_i$ 将不具备单调性

$$dp[k] = (S + \text{sum}T_i) \times \text{sum}C_k + dp[i] - S \times \text{sum}C_n + \text{sum}T_i \times \text{sum}C_i$$

由于单调性不满足，因此不能弹出队头，要维护整个凸壳。凸壳具有下凸性，最优决策点的斜率 $\text{sum}T[i]+s$ 满足：

$K_{\text{左侧}} < \text{sum}T_i + S < K_{\text{右侧}}$ ，可借助二分寻找决策点。

思路：利用一个单调栈维护整个下凸壳，单调栈内的斜率是递增的，每次二分出最后一个比决策直线斜率小的线段，其右端点即为答案。

动态规划进阶应用

【题目07】[SDOI2012] 任务安排 (P5785)

核心代码实现

```
6 int search(int i){
7     int l=1,r=top,ans=0;
8     while (l<=r){
9         int mid=(l+r)>>1,a=sta[mid-1],b=sta[mid];
10        if(dp[b]-dp[a]<=(sumT[i]+s)*(sumC[b]-sumC[a]))
11            l=mid+1,ans=mid;
12        else
13            r=mid-1;
14    }
15    return sta[ans];
16 }
17 signed main(){
18     cin>>n>>s;
19     for(int i=1;i<=n;i++)
20         cin>>t[i]>>p[i];
21     for(int i=1;i<=n;i++){
22         sumT[i]=sumT[i-1]+t[i];
23         sumC[i]=sumC[i-1]+p[i];
24     }
25     memset(dp,0x3f,sizeof dp);
26     sta[top=1]=dp[0]=0;
27     for (int i=1;i<=n;i++){
28         int k=search(i);
29         dp[i]=dp[k]+s*(sumC[n]-sumC[k])+sumT[i]*(sumC[i]-sumC[k]);
30         while(top>1&&(dp[sta[top]]-dp[sta[top-1]]*(sumC[i]-sumC[sta[top]])>=(dp[i]-dp[sta[top]]*(sumC[sta[top]]-sumC[sta[top-1]])))
31             top--;
32         sta[++top]=i;
33     }
34     cout<<dp[n]<<'\\n';
```

动态规划进阶应用

推荐题目

序号	题目	备注
1	[NOIP 2018 普及组] 摆渡车 (P5017)	
2	[ZJOI2007] 仓库建设 (P2120)	两项单调
3	[HNOI2008] 玩具装箱 (P3195)	两项单调
4	[USACO08MAR] Land Acquisition G (P2900)	两项单调
5	[APIO2014] 序列分割 (P3648)	两项单调
6	[CEOI 2004] 锯木厂选址 (P4360)	两项单调
7	[APIO2010] 特别行动队 (P3628)	两项单调
8	[SDOI2016] 征途 (P4072)	两项单调
9	[CEOI 2017] Building Bridges (P4655)	$X(j)$ 与 $k0[i]$ 均不单调)
10	高速公路 (P3394)	$X(j)$ 单调 $k0[i]$ 不单调, 树上转移)
11	[NOI2007] 货币兑换 (P4027)	($X(j)$ 与 $k0[i]$ 均不单调)
12	[NOI2014] 购票 (P2305)	($X(j)$ 单调 $k0[i]$ 不单调。)
13	[NOI2016] 国王饮水记 (P1712)	两项单调